
Do Tabular Foundation Models Scale with Dimensionality?

A Synthetic-Data Study from 10 to 10,000 Features

Timothy Daley¹

Abstract

Tabular foundation models such as TabPFN v2, TabICL, and Nori apply in-context learning to tabular regression and classification, but most released models have a hard cap on the number of features they were pretrained and validated on (500 for TabPFN v2, roughly 2,000 for TabICL; Nori publishes no such cap). We study how three such models behave as the number of features D grows from 10 to 10,000 under two conditions that prior high-dimensional tabular benchmarks confound: a **fixed-signal** condition, where a 5-feature nonlinear true function is embedded in D ambient features (the rest noisy functions of the true features or pure noise) so ambient dimensionality grows while true signal complexity does not; and a **scaling-signal** condition, where the number of causal features itself grows with D (5% of D), so both grow together as in most real high-dimensional data. Beyond a model’s native feature cap we apply a feature-bagging ensemble (fit independent copies on disjoint feature chunks, average predictions), the standard workaround from prior work; Nori, having no such cap, is run natively throughout. Under the fixed-signal condition, all three foundation models hold up: TabICL’s held-out test R^2 stays within 0.899–0.937, TabPFN within 0.897–0.933, and Nori within 0.928–0.950, across three orders of magnitude in D , and all three clearly dominate Ridge, Random Forest, and XGBoost throughout – no degradation. Under the scaling-signal condition, with training set size held fixed at 2000 rows, every model degrades sharply, but not equally: TabICL falls from $R^2=0.937$ at $D=10$ to 0.151 at $D=10000$ (an 84% relative drop, and the best of the six models at that largest D , matched by

a plain cross-validated Ridge baseline at 0.157), Nori falls to 0.098, and TabPFN – the model whose native regime the feature-bagging ensemble was originally proposed to extend – falls furthest of all six models tested, to 0.035, below Random Forest (0.061) and XGBoost (0.082). Dimensionality alone is not the hazard; dimensionality that carries proportionally more true signal than a fixed training set can support is, and different foundation model architectures degrade at very different rates once it does. TabPFN’s default model version is gated behind an interactive, human-completed license acceptance, but we found its original v2 weights are available ungated directly from Hugging Face, which let us obtain full automated results for all three foundation models; we describe this workaround so others facing the same blocker do not need to rediscover it.

1. Introduction

Tabular foundation models – transformer architectures pretrained on large corpora of synthetic or real tabular datasets and applied zero-shot via in-context learning – have shown striking accuracy on small- to medium-sized tabular regression and classification benchmarks (Müller et al., 2022; Hollmann et al., 2025). Unlike gradient-boosted trees or linear models, these models are not refit per dataset; instead, the training set is passed as context alongside the query points, and a single forward pass produces predictions.

This convenience comes with an architectural constraint that has received little systematic study: most released tabular foundation models have a hard cap on the number of features and rows they were pretrained and validated on. TabPFN v2 (Hollmann et al., 2025) was trained and evaluated up to 500 features and 10,000 rows; TabICL (Qu et al., 2025) extends this considerably but still caps out in the low thousands of features. Nori¹ publishes no analogous hard feature-count cap, though its architecture’s own sample-attention cost still grows with row count. Real tabular data routinely exceeds

¹Independent research. Correspondence to: Timothy Daley <timy.pdaley@gmail.com>.

¹<https://huggingface.co/SyntheFY/Nori>

these limits – genomics, click-stream, and sensor-fusion datasets can have thousands to tens of thousands of columns – so understanding *how* these models degrade (or don’t) past their trained regime, and whether standard workarounds close the gap, is a practical question for anyone considering a tabular foundation model outside a benchmark setting.

We study this question with a controlled synthetic data generating process, under two conditions that most real high-dimensional tabular benchmarks conflate. In the **fixed-signal** condition, a fixed, low-dimensional (5-feature) non-linear function determines the regression target, and the remaining $D - 5$ ambient features are either noisy non-linear functions of the true features or pure noise, with D swept on a log-spaced grid from 10 to 10,000. Because the true signal’s dimensionality is fixed while the ambient dimensionality grows, this isolates the effect of dimensionality *per se* from the effect of adding more true signal. But real high-dimensional tabular data rarely works this way: a genomics dataset with 10,000 measured genes, or a click-stream dataset with 10,000 engineered features, typically has more true predictive structure than a 10-feature version of the same problem, not the same 5 causal factors buried deeper in noise. We therefore also run a **scaling-signal** condition, where the number of causal features itself grows with D (Section 3.1), holding training set size fixed – the harder, arguably more realistic regime, and the one under which we find real degradation (Section 5).

We implemented the same evaluation for TabPFN v2, TabICL, and Nori – the two capped models wrapped in a feature-bagging ensemble (Feuer et al., 2023) once D exceeds their native cap, Nori run natively throughout since it has no such cap – against Ridge regression, Random Forest (Breiman, 2001), and XGBoost (Chen & Guestrin, 2016), under both a fixed training-set-size regime and a regime where training set size scales with D . TabPFN v2’s default model version requires a one-time, human-completed license acceptance with the model provider before its weights can be downloaded; we found this is specific to that default version, not to TabPFN v2’s model weights in general, and describe a workaround (Section 4) that let us obtain full results for all three foundation models rather than report this as an unresolved blocker.

Our contributions are:

- A synthetic benchmark generator with a controllable nonlinear true signal embedded in up to 10,000 ambient features, supporting both a fixed and a D -scaling number of causal features, released with this paper.
- An empirical characterization of how held-out test RMSE/ R^2 for three tabular foundation models – TabPFN v2, TabICL, and Nori – scales with D , relative to tree and linear baselines, under fixed- N and

N -scaling-with- D training regimes, when the number of causal features is held fixed at 5.

- A second characterization under a scaling-signal condition, showing that when the number of causal features grows with D instead, every model degrades sharply but by very different amounts – TabPFN degrades furthest of all six models tested, more than the plain tree and linear baselines, while TabICL degrades least – isolating dimensionality growth alone (not harmful, in our first result) from dimensionality growth that outpaces a fixed training set’s ability to resolve proportionally more true structure (which is, and not equally for every architecture).
- A direct empirical test of the feature-bagging ensemble workaround (Feuer et al., 2023) at the specific scale (10,000 features) it was proposed to address, under both conditions, and a comparison against Nori, a model requiring no such workaround, that helps isolate the ensemble’s own contribution to the scaling-signal degradation from a shared statistical difficulty affecting every model regardless of chunking.
- A documented workaround for TabPFN v2’s license-acceptance gate – its original v2 weights are available directly and ungated from Hugging Face – that let us obtain full automated results rather than report this as a blocker, released for others facing the same gate.

2. Related Work

Tabular foundation models. Müller et al. (2022) introduced Prior-Data Fitted Networks (PFNs), transformers trained on synthetic data to approximate Bayesian inference in a single forward pass, which Hollmann et al. (2025) scaled into TabPFN v2, a model pretrained once and applied zero-shot to new tabular datasets via in-context learning, achieving Nature-cover accuracy on datasets up to 10,000 rows and 500 features. Qu et al. (2025) proposed TabICL, which restructures the in-context learning architecture (a column-then-row transformer with a fixed-size latent summary) specifically to scale to far larger row counts and a higher feature cap. Ma et al. (2024) (TabDPT) and Zeng et al. (2025) (TabFlex) pursue related scaling directions – retrieval-augmented real-data pretraining, and linear-attention mechanisms, respectively – motivated by the same observation we study directly: that naively applying an in-context tabular learner outside its trained regime degrades performance. Nori (Synthefy) is a smaller (5.9M-parameter) in-context regressor that alternates feature- and sample-attention layers and, unlike TabPFN v2 and TabICL, publishes no hard feature-count cap; we include it as a third, architecturally distinct foundation model that requires no feature-bagging workaround at any D in our grid, which lets

us test whether the ensemble wrapper itself, rather than the underlying task difficulty, drives the scaling-signal degradation (Section 6).

Scaling TabPFN past its trained regime. Feuer et al. (2023) is the paper most directly relevant to our method: they show TabPFN’s accuracy falls as feature count grows past its trained regime, and propose closing the gap with sketching, feature selection, and feature-bagging ensembles – fitting independent copies of TabPFN on random feature subsets and averaging. We adopt their feature-bagging approach as the mechanism for applying TabPFN v2 and TabICL beyond their native caps, and, unlike their work (which evaluates on existing high-dimensional tabular benchmarks, where the number of truly informative features is unknown and grows with D), we hold the number of truly informative features fixed at 5 while sweeping D , isolating the effect of ambient dimensionality from the effect of additional signal.

High-dimensional tabular learning more broadly. Feature selection, random projections, and ensembling over feature subsets are classical tools for high-dimensional tabular learning with tree and linear models; our contribution is not a new such tool, but an empirical measurement of how well the existing tool (feature-bagging) closes the dimensionality gap for the specific architectures that motivated it.

3. Method

3.1. Synthetic data generating process

Each dataset is parameterized by an ambient feature count D , a causal feature count $D_{\text{true}} \leq D$, and a sample count N . We draw D_{true} latent features $z = (z_1, \dots, z_{D_{\text{true}}}) \sim \mathcal{N}(0, I_{D_{\text{true}}})$ per row and define the regression target through a fixed nonlinear function $f(z)$:

$$y = f(z) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, (\eta \cdot \sigma_f)^2), \quad (1)$$

where σ_f is the empirical standard deviation of $f(z)$ over the batch and η (the noise-to-signal ratio) is fixed across all D so the achievable ceiling on test R^2 does not itself vary with dimensionality holding D_{true} fixed. For $D_{\text{true}} = 5$, f is the fixed hand-written function

$$f(z) = 3 \sin(z_1 z_2) + 2z_3^2 - 1.5 z_4 z_5 + z_1 z_4^2 - 0.5 z_2^3 + 2 \tanh(z_1 + z_3 - z_4). \quad (2)$$

For other D_{true} , f is a sum of $2D_{\text{true}}$ terms, each a random transform (identity, square, sine, or tanh) of a weighted combination of $k \in \{1, 2, 3\}$ causal features chosen uniformly without replacement, with random weights and an overall coefficient – the same construction used for the derived ambient features below, so causal-feature interactions

exist for the feature-bagging ensemble to potentially sever across chunks (Section 6). This term set is seeded only by D_{true} , not by the per-run data seed, so every seed sharing a given D_{true} evaluates the identical task – the seed axis varies the drawn data, not the task itself, exactly as for the hand-written $D_{\text{true}} = 5$ case.

We control D_{true} via two **signal regimes**: **fixed-signal**, where $D_{\text{true}} = 5$ regardless of D (isolating ambient dimensionality from signal dimensionality, and reproducing the hand-written f above); and **scaling-signal**, where $D_{\text{true}} = \max(5, \text{round}(0.05D))$, so 5% of features are causal, reaching $D_{\text{true}} = 500$ at $D = 10000$.

The remaining $D - D_{\text{true}}$ ambient features are split into two kinds. A fraction p_{noise} are pure noise, $x_j \sim \mathcal{N}(0, 1)$, independent of y . The rest are noisy derived features: for each, we sample $k \in \{1, 2, 3\}$ of the true latent features uniformly without replacement, a random weight vector, and one of the same four transforms applied to the weighted sum, then add independent Gaussian noise at scale ρ . These derived features are correlated with y (through z) but not perfectly recoverable, matching the informal specification that ambient features be “correlated or noisy functions of the true features...but not 100% derivable.” All D columns (true, derived, and pure-noise) are then randomly permuted so column position carries no information. Train, validation, and test splits are drawn as one i.i.d. block per (D, N, seed) and sliced, since rows are independent draws.

3.2. Dimensionality and sample-size grid

We sweep $D \in \{10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000\}$.

Two regimes control the training set size N_{train} : **fixed-N**, where $N_{\text{train}} = 2000$ is held constant across all D ; and **scaling-N**, where $N_{\text{train}} = \text{clip}(4D, 200, 2000)$, i.e. training set size grows linearly with D until it reaches the same 2000-row ceiling used by the fixed-N regime. This shared ceiling is set by our compute budget: on the CPU-only machine used for these experiments (34GB RAM, no CUDA), a single TabICL chunk at $N_{\text{train}} = 2000$ and its 2000-feature cap was the largest size we could run without the process being killed for exceeding available memory (Section 4). Validation and test sets (500 and 1000 rows respectively) are held fixed across all conditions so evaluation is comparable. Each cell is repeated over 5 random seeds.

The fixed-signal condition is run under both N -regimes. The scaling-signal condition is run only under fixed- N ($N_{\text{train}} = 2000$ for every D): this is also a compute-budget choice, and it is the harder of the two N -regimes for scaling-signal, since the amount of training data does not grow to compensate for the growing amount of true structure – if degradation were absent even here, it would have been hard to argue for it under scaling- N either.

3.3. Models

We evaluate Ridge regression (Hoerl & Kennard, 1970), Random Forest (Breiman, 2001), XGBoost (Chen & Guestrin, 2016), TabPFN v2 (Hollmann et al., 2025), TabICL (Qu et al., 2025), and Nori. For TabPFN v2 and TabICL, whenever D exceeds the model’s native feature cap ($c = 500$ for TabPFN v2, $c = 2000$ for TabICL), we apply a feature-bagging ensemble (Feuer et al., 2023): partition the D columns into $\lceil D/c \rceil$ disjoint chunks of size $\leq c$ via a random permutation, fit an independent copy of the model on each chunk, and average the chunks’ predictions. When $D \leq c$ this reduces to a single chunk containing every feature, i.e. the model runs exactly as it natively would. Nori publishes no analogous feature-count cap – its documented constraint is attention cost over rows, not columns, and N_{train} is capped at 2000 throughout our grid regardless of D (Section 3.2) – so it is run natively across the entire D grid with no ensemble wrapper.

4. Experimental Setup

All experiments ran on a single Apple Silicon (M1 Max, 34GB RAM) machine. Ridge, Random Forest, XGBoost, TabICL, and Nori ran CPU-only – no CUDA GPU was available on this machine. TabPFN, however, ran on the machine’s Apple GPU via PyTorch’s Metal Performance Shaders (MPS) backend, which gave a real $\sim 6\times$ speedup over CPU in our timing checks (e.g. 155s $\rightarrow$ 25s for a single fit at $D=2000$) with no accuracy difference; we used it for all TabPFN results below. We initially tried MPS for Nori too, but repeated model instantiations within one long-running sweep process leaked GPU memory with no release between calls and reliably exhausted the shared MPS memory pool after roughly 25 calls, despite Nori showing a comparable $\sim 3\text{--}4\times$ MPS speedup when it did not fail; we therefore pin Nori to CPU unconditionally for reliability over the multi-hour sweep, at the cost of that speedup. TabPFN did not exhibit this failure mode in the same length of run. None of these models’ primary authors optimize for or validate on Apple GPUs, so none of these wall-clock numbers should be read as representative of production latency. Software versions: Python 3.12 (native arm64), torch 2.13.0, tabpfn 8.5.0, tabicl 2.1.1, syntheyfy-nori 0.20.0, scikit-learn 1.9.0, xgboost 3.4.1, numpy 2.5.2.

For TabPFN we pass `ignore_pretraining_limits=True` so a chunk that exceeds an internal feature-count threshold degrades gracefully rather than raising. Both TabPFN and TabICL self-ensemble over multiple internal estimators by default (TabPFN: `n_estimators='auto'`, scaled by dataset size; TabICL: 8); on our hardware the default configuration was killed by the OS for exceeding available memory on a single TabICL fit at $D=2000$, $N_{\text{train}}=2000$.

We therefore fix `n_estimators=1` for both foundation models throughout, which keeps memory bounded and was substantially faster in our timing checks. This is a compute-constrained choice: it should not be read as a claim that either model’s best achievable accuracy is reported here, only their accuracy under this fixed compute budget, on equal footing with each other.

TabPFN v2’s license gate has a workaround. The `tabpfn` package’s default model version (v2.5 and later) requires downloading weights from PriorLabs, gated behind a one-time license acceptance completed interactively in a browser, followed by an API token generated from an account page – a human-in-the-loop step with no automated or headless path around it, which an earlier draft of this paper reported as an unresolved blocker. We subsequently found this gate is specific to those default (v2.5+) model versions: the original TabPFN v2 weights (`Prior-Labs/TabPFN-v2-reg` on Hugging Face) are not gated at all – a plain, unauthenticated `huggingface_hub` download, no login and no token required, confirmed to succeed with zero interactive steps in an environment with no browser or display available. Setting the environment variable `TABPFN_MODEL_VERSION=v2` before the package’s first import selects these ungated weights, which is what we use throughout this paper. We flag this because it is easy to conclude, as we initially did, that TabPFN v2 cannot be evaluated without human intervention at all; that conclusion is specific to its newer default weights, not to the model.

Nori requires no license step. Nori’s checkpoint is Apache-2.0 licensed and downloads automatically and ungated from Hugging Face on first use, with no token or gated-repo acceptance of any kind.

5. Results

5.1. Fixed-signal condition: ambient dimensionality alone

Figure 1 shows held-out test R^2 as a function of D for both N -regimes under the fixed-signal condition ($D_{\text{true}} = 5$ regardless of D), averaged over 5 seeds (error bars: ± 1 SD), for all six models.

None of the three foundation models degrade with dimensionality in our setting, and all three clearly dominate every baseline throughout. In the fixed- N regime, TabICL’s mean test R^2 starts at 0.937 ($D = 10$), dips to a minimum of 0.899 around $D = 200\text{--}500$, and recovers to 0.928 by $D = 10000$ – a span of at most 0.038 across three orders of magnitude in D , with no monotonic trend. TabPFN and Nori show the same qualitative pattern: TabPFN ranges from 0.897 (its minimum, at $D = 500$) to 0.933 ($D = 10/D = 20$, tied), a span of 0.036; Nori ranges

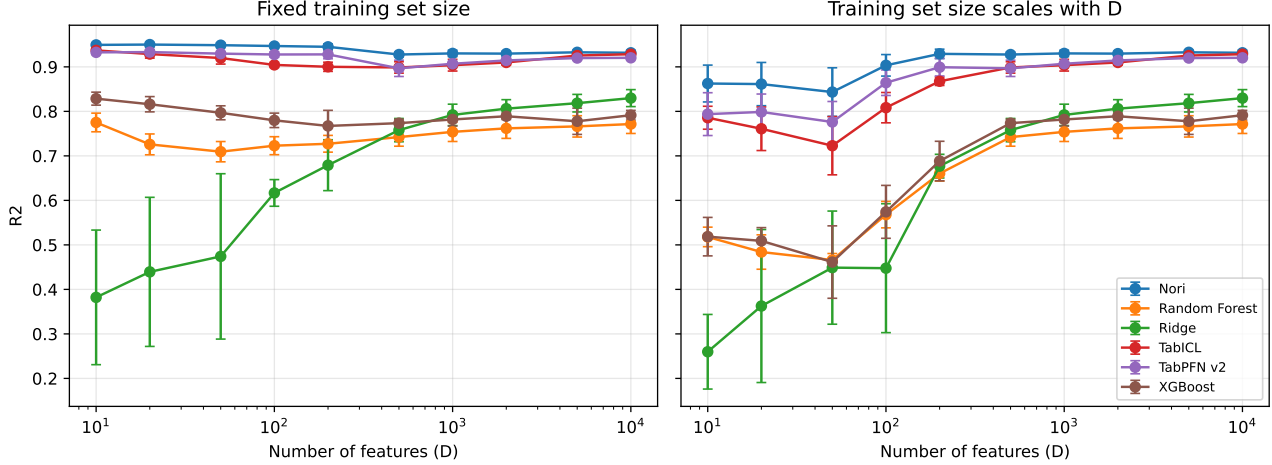


Figure 1. Held-out test R^2 vs. number of features D (log scale), for training set size held fixed (left) and scaling with D (right). Points are means over 5 seeds; error bars are ± 1 SD. TabICL uses the feature-bagging ensemble beyond its 2000-feature cap ($D = 5000$: 3 chunks; $D = 10000$: 5 chunks); TabPFN beyond its 500-feature cap ($D = 1000$: 2 chunks; $D = 5000$: 10 chunks; $D = 10000$: 20 chunks); Nori has no cap and is never chunked.

from 0.928 (also at $D = 500$) to 0.950 ($D = 20$), a span of 0.022 – the narrowest of the three. All three foundation models dip at the same D (500), the boundary where derived features begin outnumbering pure-noise ones by the largest margin in our construction (Section 3.1); we do not have a mechanistic explanation for why this particular point is shared across three otherwise different architectures and flag it as an open question. Crucially, this includes the largest D values, where TabICL and TabPFN are running as multi-chunk feature-bagging ensembles rather than natively (5 and 20 chunks respectively at $D = 10000$): the ensemble does not visibly cost accuracy relative to each model’s own native-scale dip. Every baseline sits well below all three foundation models across the entire grid: at $D = 10000$ (fixed- N), XGBoost reaches 0.791, Random Forest 0.771, and RidgeCV 0.830 – the closest baseline, but still 0.098 below TabICL and further still below TabPFN and Nori.

In the scaling- N regime, all six models start lower at small D (where N_{train} is also small: 200 rows at $D = 10$) and climb as D (and hence N_{train}) grows, converging exactly with the fixed- N numbers once N_{train} saturates at its 2000-row ceiling for $D \geq 500$ (by construction, Section 3.2). All three foundation models lead this climb throughout, with Nori consistently at or ahead of the other two: at $D = 10$ ($N_{\text{train}} = 200$), Nori already reaches 0.863, ahead of TabPFN’s 0.794 and TabICL’s 0.786; by $D = 10000$ ($N_{\text{train}} = 2000$) the three have converged to within 0.012 of each other (Nori 0.932, TabICL 0.928, TabPFN 0.920). All three stay well clear of XGBoost (0.519 $\rightarrow$ 0.791) and RidgeCV (0.260 $\rightarrow$ 0.830) over the same range.

RidgeCV is the weakest model at low D (0.382 at $D = 10$,

fixed- N) but improves monotonically with D and overtakes both tree baselines by $D = 10000$ (0.830 vs. 0.791 XGBoost, 0.771 Random Forest). We discuss why in Section 6. Random Forest and XGBoost are both roughly flat across D in the fixed- N regime (0.71–0.83), consistent with tree ensembles’ typical robustness to irrelevant or weakly-informative features via per-split feature subsampling.

RMSE results (Figure in figures/rmse_vs_dimensionality.pdf, summary table in figures/rmse_summary.csv) tell the same story and are omitted here for space.

5.2. Scaling-signal condition: dimensionality that carries more signal

Figure 2 places the fixed-signal condition above (left) next to the scaling-signal condition (right), both under the fixed- N regime ($N_{\text{train}} = 2000$ throughout), so the only difference between panels is whether D_{true} stays at 5 or grows to $\max(5, 0.05D)$ alongside D .

Every model degrades sharply once causal dimensionality scales with D , but not equally – TabPFN degrades furthest of all six, TabICL least. The two panels are identical at $D \in \{10, 20, 50, 100\}$, where the scaling-signal rule still floors at $D_{\text{true}} = 5$ (Section 3.1) – a useful internal check, since the two conditions are literally the same task there and the numbers match exactly for every model. They diverge sharply from $D = 200$ ($D_{\text{true}} = 10$) onward, and the three foundation models – indistinguishable from each other under fixed-signal – separate sharply from one another for the first time in this paper. TabICL falls from $R^2 = 0.937$

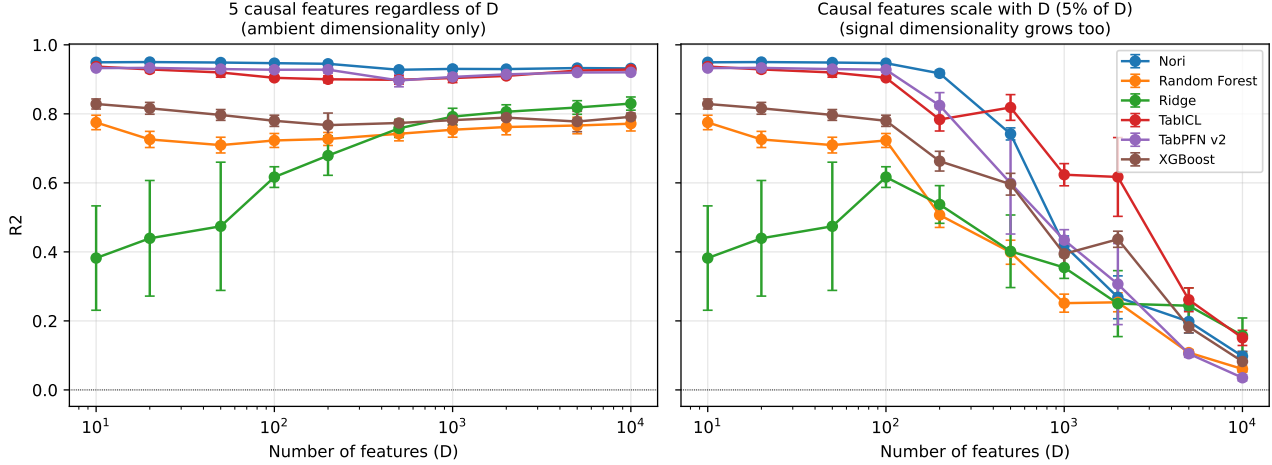


Figure 2. Held-out test R^2 vs. D under fixed- N ($N_{\text{train}} = 2000$), for the fixed-signal (left, $D_{\text{true}} = 5$ always) and scaling-signal (right, $D_{\text{true}} = \max(5, 0.05D)$) conditions. Points are means over 5 seeds; error bars are ± 1 SD.

at $D = 10$ to 0.784 at $D = 200$, 0.624 at $D = 1000$, and 0.151 at $D = 10000$ ($D_{\text{true}} = 500$) – an 84% relative drop. Nori falls from 0.949 to 0.917, 0.419, and 0.098 over the same range – a steeper 90% relative drop, despite running natively with no feature-bagging ensemble at any D in our grid. TabPFN falls furthest of all: from 0.933 to 0.824, 0.434, and 0.035 – a 96% relative drop, ending below every other model in this study, foundation or baseline. XGBoost and Random Forest show broadly the same qualitative pattern (XGBoost: 0.829 $\rightarrow$ 0.082; Random Forest: 0.775 $\rightarrow$ 0.061), and even RidgeCV, which was improving with D under fixed-signal, now falls (0.382 $\rightarrow$ 0.157, after a partial recovery at $D = 100$ –200 where more causal features briefly help a linear model directly).

At the largest D , the foundation model the feature-bagging ensemble was originally designed to extend performs worst of all six models – including the plain linear baseline. At $D = 1000$ the three foundation models still lead clearly (TabICL 0.624, TabPFN 0.434, Nori 0.419, versus XGBoost’s 0.394, RidgeCV’s 0.355, Random Forest’s 0.251). By $D = 10000$ the ordering inverts sharply for TabPFN specifically: RidgeCV (0.157) and TabICL (0.151, statistically indistinguishable from RidgeCV) are the best two models; Nori (0.098) sits in the middle, still ahead of XGBoost (0.082) and Random Forest (0.061); and TabPFN (0.035) is the single worst model of the six, below both tree baselines and the plain linear model it was designed to outperform. This is despite the feature-bagging ensemble (Feuer et al., 2023) being exactly the mechanism proposed to let TabPFN operate past its native 500-feature cap – at $D = 10000$ it is running as a 20-chunk ensemble, the finest chunking of any model in this study (TabICL uses 5 chunks at the same D ; Section 6 discusses a candidate explanation,

and Nori’s comparably severe collapse with zero chunking argues against chunking being the whole story). Whatever the mechanism, TabICL’s and TabPFN’s dominance in Section 5.1 does not carry over unconditionally – it holds when the amount of true structure is small and fixed, and both erodes and reorders sharply as the task’s true complexity grows toward (and past) what 2000 training rows can resolve.

6. Discussion and Limitations

What causes the scaling-signal degradation? Three candidate mechanisms, not mutually exclusive. (1) *Statistical difficulty*: with N_{train} fixed at 2000, resolving $D_{\text{true}} = 500$ interacting causal features from 2000 rows is a much harder estimation problem than resolving 5, independent of which model is used. Nori gives us direct evidence that this mechanism operates on its own: Nori applies no feature-bagging ensemble at any D in our grid, yet still collapses by 90% (relative) at $D=10000$ – comparable to, and by this relative measure worse than, TabICL’s chunked 84% drop, despite zero chunking. Random Forest and XGBoost, likewise never chunked, degrade by similarly large amounts. This is strong evidence that mechanism (1) alone is sufficient to produce large degradation, with no ensemble wrapper required. (2) *Severed interactions under feature-bagging*: TabICL’s and TabPFN’s ensembles randomly partition D columns into disjoint chunks of size $\leq c$ ($c = 2000$ and $c = 500$ respectively); at $D = 10000$, two causal features assigned independently and uniformly at random land in the same TabICL chunk (5 chunks) with probability $\approx 1/5$, but in the same TabPFN chunk (20 chunks) with probability only $\approx 1/20$ – an interacting term is roughly $4\times$ more likely to be split across chunks for TabPFN than for TabICL at this D .

This is a plausible contributor to TabPFN degrading furthest of all six models (Section 5.2). It is not, on its own, a complete explanation, though: Random Forest and XGBoost, never chunked at all, degrade by amounts comparable to or larger than TabPFN’s, so chunking granularity alone does not explain the full six-model ranking. (3) *Derived-feature dilution*: under scaling-signal there are also more *derived* ambient features referencing the (now larger) causal set, diluting the true columns’ relative share of D similarly in both conditions – this alone does not obviously explain the divergence, since the fixed-signal condition has the same 95%-ambient structure and does not degrade. Taken together, we read mechanism (1) as the primary driver across all six models – Nori’s unchunked collapse is the clearest single piece of evidence for this in our design, something the original TabICL-only version of this study could not establish – with mechanism (2) plausibly compounding it specifically for TabPFN’s unusually fine chunking, on top of whatever architecture-specific factors separate TabICL’s, TabPFN’s, and Nori’s differing degradation rates that this design cannot isolate. Cleanly separating (1) from (2) for TabICL and TabPFN specifically would still require running each natively (unchunked) at $D_{\text{true}} = 500$, which exceeds both their feature caps by construction at the only D in our grid where D_{true} reaches 500; we flag this decomposition as future work.

Why RidgeCV overtakes the tree baselines at high D under the fixed-signal condition. (This effect is specific to fixed-signal; RidgeCV does not overtake the trees under scaling-signal, Section 5.2.) This is very likely an artifact of our specific data generating process rather than a general property of linear models in high dimensions. Each derived feature (Section 3.1) is a noisy nonlinear transform (square, sine, or tanh) of a small weighted combination of the true latent features. As D grows, more such derived features are drawn, and by chance some land close to a term that actually appears in $f(z)$ (e.g. a transform close to z_3^2 or $\tanh(\cdot)$ of a similar combination). Once enough approximately-matching nonlinear basis functions accumulate among the D columns, a purely linear model over those columns can partially reconstruct the nonlinear $f(z)$ – effectively an unintentional random-features expansion. This is a property of our specific derived-feature construction (Section 3.1), not evidence that linear models generically improve with ambient dimensionality; a construction using a single fixed transform family, or none, would likely not produce this effect.

An unplanned validation of the benchmark against known theory. During development, plain Ridge regression at a fixed $\alpha = 1$ produced a strikingly bad result ($R^2 \approx -0.5$ to -1.0) at exactly $D = N_{\text{train}} = 2000$ in the fixed- N regime, with normal-looking results on both sides. We initially suspected a data generation or solver

bug, but multiple solvers (SVD, Cholesky, LSQR) agreed on the value, and the training design matrix was full rank. This is the double-descent phenomenon (Belkin et al., 2019): weakly-regularized linear models are known to peak in test error exactly at the interpolation threshold $n = p$, with generalization recovering on both the classical ($n \gg p$) and overparameterized ($p \gg n$) sides. That our synthetic benchmark reproduces this exact, sharply localized, theoretically predicted failure mode – at precisely the one D value where N_{train} crosses it – is a useful sanity check that the pipeline generates and evaluates data correctly. We report all baselines using RidgeCV (cross-validated α) rather than fixed- α Ridge, since practitioners cross-validate in practice and an artificially bad linear baseline at one specific D would otherwise distract from the paper’s actual question.

The feature-bagging ensemble is our addition, not a native capability. Every result past a model’s native feature cap (TabPFN v2: 500; TabICL: 2000) characterizes *our wrapper* applied to that model, not the released model in isolation; Nori has no native cap and so is never subject to this caveat. Under fixed-signal, both TabICL and TabPFN show no visible accuracy cost moving from native scale to, respectively, a 5- and 20-chunk ensemble at $D = 10000$, evidence the wrapper is a reasonable workaround when causal features are few enough that chunking rarely splits an interaction across chunks. Under scaling-signal, where $D_{\text{true}} = 500$ at that same $D = 10000$, the wrapper’s chunking plausibly compounds the degradation for both models (Section 6, mechanism 2), more severely for TabPFN’s finer 20-chunk partition than TabICL’s coarser 5-chunk one – though Nori’s comparably severe collapse with zero chunking (Section 6, mechanism 1) shows the wrapper is not required to produce large degradation, only a plausible compounding factor. We cannot rule chunking in or out cleanly with the current design, since testing TabICL or TabPFN without chunking at $D_{\text{true}} = 500$ requires $D \leq 2000$ and $D \leq 500$ respectively, where the scaling-signal rule has not yet reached $D_{\text{true}} = 500$.

Compute constraints. All experiments ran on a single Apple Silicon machine (M1 Max, 34GB RAM, no CUDA GPU); TabPFN ran via the machine’s Apple GPU (MPS backend), while every other model ran CPU-only (Section 4). None of these models’ authors optimize for or validate on this hardware, so absolute wall-clock numbers here are not representative of production latency, and we were forced to fix `n_estimators=1` for TabPFN and TabICL to stay within available memory, which likely understates their best achievable accuracy at any given D . We also found Nori leaks GPU memory across repeated instantiations within one long-running process and pinned it to CPU for reliability (Section 4), giving up a measured $\sim 3\text{--}4\times$ speedup for the same reason; its reported accuracy is unaffected by this choice of device.

Licensing. TabPFN v2 and later PriorLabs checkpoints are released under a non-commercial research license; TabICL is BSD-3-Clause; Nori is Apache-2.0.

Scope of the generative model. A single fixed nonlinear true function family and one specific derived-feature construction were used throughout; results characterize this generative family, not dimensionality scaling in general. The scaling-signal rule (5% of D is causal) is one arbitrary choice among many plausible ones; we do not claim this fraction is representative of any particular real domain, only that *some* such fraction is more realistic than holding D_{true} fixed at 5 while D reaches 10,000. Likewise the RidgeCV-overtakes-trees effect under fixed-signal (Section 6) is a direct consequence of the specific derived-feature construction and should not be over-generalized.

7. Conclusion

We built a synthetic benchmark that can hold ambient feature dimensionality and causal signal dimensionality either independent or coupled, and used it to test whether three tabular foundation models’ accuracy degrades as D grows past their trained regime. The answer depends entirely on which of those two quantities is actually growing, and, once it does, on which specific model is being asked to handle it. When only ambient dimensionality grows (fixed-signal, $D_{\text{true}} = 5$ throughout), none of the three degraded: test R^2 stayed within a narrow band for each – TabICL 0.899–0.937, TabPFN 0.897–0.933, Nori 0.928–0.950 – across three orders of magnitude in D , including past TabICL’s and TabPFN’s native feature caps under a feature-bagging ensemble, and all three clearly dominated Ridge, Random Forest, and XGBoost baselines throughout. When causal dimensionality scales with D instead (scaling-signal, training set size held fixed), every model degraded sharply, but by very different amounts: TabICL fell least, from $R^2 = 0.937$ at $D = 10$ to 0.151 at $D = 10000$, matched there by a plain cross-validated Ridge baseline; Nori fell to 0.098; and TabPFN – the model whose native regime the feature-bagging ensemble was originally designed to extend – fell furthest of all six models tested, to 0.035, below both tree baselines. Dimensionality growth alone was not the hazard in our experiments; dimensionality growth that outpaces what a fixed amount of training data can resolve was, and different foundation model architectures absorb that hazard very differently, with the most heavily-chunked model fairing the worst. Real high-dimensional tabular data is presumably closer to our scaling-signal condition than our fixed-signal one, which is a reason for caution before reading robustness results like our own fixed-signal finding as license to deploy a tabular foundation model on arbitrarily wide data without checking whether more features are also bringing more (or less) signal per row of training data – and,

per our results, without checking which specific model is being deployed.

An earlier version of this paper reported TabPFN v2 as unrunnable, since its default model version gates its weights behind an interactive license-acceptance step with no automated path around it. We subsequently found this gate is specific to that default version, not to TabPFN v2’s weights in general: its original v2 checkpoint downloads directly and ungated from Hugging Face (Section 4), which let us complete the comparison we had originally released only as an untested code path. We also added Nori, a third, architecturally distinct foundation model with no published feature cap, which turned out to matter for more than completeness: its comparably severe collapse under scaling-signal, despite using no feature-bagging ensemble at all, is the clearest evidence in this study that the scaling-signal hazard is a property of the estimation problem itself, not primarily an artifact of the ensemble wrapper used to extend TabICL and TabPFN past their native caps (Section 6). Along the way, our benchmark reproduced the double-descent phenomenon at exactly the interpolation threshold for an under-regularized linear baseline, which we take as independent evidence the pipeline is measuring what it claims to measure.

Acknowledgements

This research was assisted by Claude (Anthropic).

References

- Belkin, M., Hsu, D., Ma, S., and Mandal, S. Reconciling modern machine learning practice and the bias-variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.
- Breiman, L. Random forests. *Machine Learning*, 45(1): 5–32, 2001.
- Chen, T. and Guestrin, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- Feuer, B., Hegde, C., and Cohen, N. Scaling tabpfn: Sketching and feature selection for tabular prior-data fitted networks. *arXiv preprint arXiv:2311.10609*, 2023.
- Hoerl, A. E. and Kennard, R. W. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S. B., Schirmer, R. T., and Hutter, F. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025.

Ma, J., Thomas, V., Hosseinzadeh, R., Labach, A., Kamkari, H., Cresswell, J. C., Golestan, K., Yu, G., Caterini, A. L., and Volkovs, M. Tabdpt: Scaling tabular foundation models on real data. *arXiv preprint arXiv:2410.18164*, 2024.

Müller, S., Hollmann, N., Arango, S. P., Grabocka, J., and Hutter, F. Transformers can do bayesian inference. *International Conference on Learning Representations*, 2022.

Qu, J., Holzmüller, D., Varoquaux, G., and Le Morvan, M. Tabicl: A tabular foundation model for in-context learning on large data. *arXiv preprint arXiv:2502.05564*, 2025.

Zeng, Y., Dinh, T., Kang, W., and Mueller, A. C. Tabflex: Scaling tabular learning to millions with linear attention. *arXiv preprint arXiv:2506.05584*, 2025.